Unit 1: Introduction to Data Structures and Algorithms

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

What is a data structure?

You use them every day — without knowing their names.

Think about: a filing cabinet, a stack of cafeteria trays, a dictionary, a to-do list.

What do all of these have in common?

Data Structures

A **data structure** is a way of organizing and storing data in a computer so that you can work with it efficiently.

Just like physical containers, each data structure is designed for a specific job:

- A **filing cabinet** → organized by label, fast to find things
- A **stack of trays** → you can only take from the top
- A **dictionary** → alphabetically sorted, fast to look up

Different structures are good at different things. Choosing the right one is one of the most important decisions you make as a programmer.

How many data structures can you name?

Think about the programs you use every day — your phone, your browser, a maps app.

What structures might be hiding underneath?

The Most Common Data Structures

Structure	Real-world analogy	What it's good at
Array	Row of mailboxes (numbered 0, 1, 2, ...)	Fast access by position
Linked list	Treasure hunt (each clue points to the next)	Fast insert/delete anywhere
Stack	Stack of plates	Last-in, first-out access
Queue	Line at a ticket counter	First-in, first-out access
Hash table	Dictionary with alphabetical tabs	Ultra-fast lookup by key
Tree	Company org chart	Hierarchical relationships
Graph	Road map	Connections between things

You Already Use These Every Day

- Your browser's **back button** → stack (most recent page on top)
- A **playlist** playing songs in order → queue
- **Google Maps** finding a route → graph
- Your phone's **contacts** → structure optimized for fast search

None of this is magic. It's just the right data structure for the job.

🤔 Does it really matter which structure you pick?

You have 1 million student records. You need to find one by name.

How would you do it? How long might it take?

Why the Choice of Data Structure Matters

Unsorted array: you may have to look at every single record.
That could mean checking **1,000,000 entries**.

Hash table: you can find the right record in roughly **1 step** — regardless of how many records there are.

Same data. Same problem. The structure you choose can be the difference between a program that runs in **milliseconds** and one that takes **hours**.

🤔 What is an algorithm?

You've heard the word before. But what does it actually mean?

In your own words — what's an algorithm?

Introduction to Algorithms

An **algorithm** is a finite, step-by-step set of instructions for solving a problem.

The key word is *finite* — it must eventually end.

- A **recipe** is an algorithm: exact steps, in order, producing a result.
- A **GPS route** is an algorithm: given your location and destination, it produces a sequence of turns.

Algorithms are not magic. They are just very precise instructions.

How would you find the largest number in a list?

```
[3, 7, 1, 9, 4, 6, 2]
```

Describe the steps you'd take — in plain English.

Any assumptions necessary?

Concrete Problem: Find the Largest Number

Algorithm:

1. Assume the first number is the largest. Call it `max`.
2. Look at each remaining number in the list.
3. If the current number is greater than `max`, update `max`.
4. After checking all numbers, `max` is the answer.

```
int findMax(vector<int>& v) {
    int max = v[0];
    for (int i = 1; i < v.size(); i++)
        if (v[i] > max) max = v[i];
    return max;
}
```

Simple, unambiguous, and it terminates. That's an algorithm.

🤔 What makes an algorithm *good*?

Two algorithms both find the max. One checks every element. One somehow checks only 3.

What qualities would you want a good algorithm to have?

Properties of a Good Algorithm

- **Correct** — it always produces the right answer.
- **Efficient** — it doesn't do more work than necessary.
- **Clear** — someone else can read and understand it.
- **Finite** — it terminates for all valid inputs.

All four matter. A fast algorithm that gives wrong answers is useless. A correct algorithm that takes a year to run is also useless.

Is an algorithm the same as a program?

You write `findMax` in C++. Your friend writes it in Python.
Someone else writes it in Java.

Are they running the same algorithm?

Algorithms Are Not Programs

An algorithm is an *idea* — a logical sequence of steps.

A program is that idea written in a specific language.

The same algorithm can be implemented in C++, Python, Java, or plain English. The algorithm exists **independently of the language**.

> This distinction matters. When we analyze efficiency, we analyze the algorithm — not the C++ code.

🥚 Which came first: the data structure or the algorithm? 🐔

Imagine designing a GPS app. You need to find the shortest route between two cities.

Do you pick your algorithm first, then figure out how to store the map? Or the other way around?

Data Structures and Algorithms Are Inseparable

You cannot design a good algorithm without choosing the right data structure — and you cannot choose a data structure without thinking about what operations you need.

> "Algorithms + Data Structures = Programs" — Niklaus Wirth, 1976

They are two sides of the same coin.

Two ways to search — which is faster?

You have a **sorted** list of 1,000 numbers. You want to find 42.

Option A: Check each number one by one from the left.
Option B: Jump to the middle — is 42 higher or lower? Discard half. Repeat.

Which would you choose? Why?

Searching: Structure Enables the Algorithm

Option A — Linear search:

- Check every element, left to right
- Works on any array (sorted or not)
- Worst case: check all 1,000 elements

Option B — Binary search:

- Jump to the middle, discard half, repeat
- Much faster — but **only works on a sorted array**

Binary search is only possible because we chose a *sorted* data structure. The data structure you pick determines which algorithms are available to you.

🤔 How would you store a social network?

You have 1 billion users. Each user has friends — anywhere from 0 to thousands.

What data structure would you use to store users and their friendships?

Social Networks: Structure Determines Speed

Array of all users:

- Finding all of a person's friends requires scanning everyone — slow.

Graph (users = nodes, friendships = edges):

- Finding all of a person's friends is instant.
- Finding friends-of-friends is a traversal.
- Suggesting "people you may know" is a graph algorithm.

The data structure makes certain algorithms fast and other algorithms irrelevant.

The Design Process

When solving a real problem, always ask:

1. What **operations** do I need? *(search? insert? delete? sort?)*
2. Which **data structure** makes those operations efficient?
3. Which **algorithm** is best suited to that structure?

These three questions are the foundation of everything in this course.

🤔 Do you need to know? Does it matter?

You use `std::stack` in C++. You call `push()` and `pop()`.
You have no idea how it stores its data internally.

Abstract Data Types (ADTs)

An **Abstract Data Type (ADT)** is a specification that defines:

- **What data** it stores
- **What operations** are available
- **What behavior** those operations have

It says *nothing* about how any of this is implemented internally.

> An ADT is the **contract**. The data structure is the **implementation**.

The Vending Machine Analogy

You know:

- It stores items
- You insert money and press a button to get one
- It gives change

You do **not** know (or care) whether it uses a conveyor belt, a rotating drum, or mechanical arms inside.

The implementation detail is hidden from you. The interface is all you need.

🤔 What operations does a Stack need?

You know a stack is LIFO — last in, first out. Like a stack of plates.

What operations would you want to be able to do on a stack?

The Stack ADT

A stack follows **Last-In, First-Out (LIFO)** order.

Operation	Description
`push(item)`	Add item to the top
`pop()`	Remove and return the top item
`peek()`	Return the top item without removing it
`isEmpty()`	Return true if the stack has no items
`size()`	Return the number of items

That's the ADT. It says nothing about *how* a stack stores its items.

🤔 Can the same ADT have different implementations?

`std::stack` exists. But you could also build a stack using a linked list.

Would they behave the same? Would the calling code need to change?

Same ADT, Different Implementations

```
// Implementation 1: array-based (std::stack uses a deque)
stack<int> s;
s.push(10);
s.push(20);
cout << s.top();  // 20
s.pop();
cout << s.top();  // 10
```

```
// Implementation 2: linked list (conceptually)
LinkedStack<int> s;
s.push(10);
s.push(20);
cout << s.top();  // 20 — identical behavior
```

The **caller's code doesn't change** when you swap implementations. That's the power of the ADT.

🤔 Why do we care about the ADT / implementation separation?

Suppose you build a huge application using `std::stack`.
Later, you find a faster stack implementation.

What would you have to change in your application code?

Why ADTs Matter

- **Separation of concerns** — the user of a data structure doesn't need to know how it works internally.
- **Interchangeability** — swap one implementation for a faster one without rewriting anything else.
- **Testability** — verify correctness by checking behavior against the ADT's contract.
- **In C++** — ADTs map naturally onto classes: public interface = ADT, private members = implementation.

> If you code to the ADT, not the implementation, your code is insulated from change.

Applications of ADTs

For each scenario, think about which ADT fits best.

🌐 Scenario 1

Your web browser keeps track of every page you visit so the back button always takes you to the previous page.

What ADT would you use?

What Doesn't Work Well

Array — you'd need to know how many pages the user will visit in advance. No good.

Queue (FIFO) — gives you the *oldest* page first. But the back button needs the *most recent* page, not the first one you ever visited.

Hash table — fast lookup by key, but there's no "order." You can't ask a hash table for "the previous item."

Browser History → Stack

Each page visit is **pushed** onto the stack.

The back button **pops** the top — giving you the most recent page.

```
Visit A → [A]
Visit B → [A, B]
Visit C → [A, B, C]
Back    → [A, B]     ← C returned, now at B
Back    → [A]        ← B returned, now at A
```

Why Stack? You always need the *most recently added* item. That's LIFO.

🖨️ Scenario 2

A shared office printer receives jobs from multiple computers. Jobs should print in the order they were sent.

What ADT would you use?

What Doesn't Work Well

Stack (LIFO) — the most recently submitted job would print first. The person who submitted first waits the longest. Chaos. 😅

Hash table — no inherent ordering. You couldn't determine which job was "first."

Tree — trees model hierarchical relationships, not sequential ordering of requests.

Print Queue → Queue

Each job is **enqueued** when submitted. The printer **dequeues** from the front.

```
Submit A → [A]
Submit B → [A, B]
Submit C → [A, B, C]
Print    → [B, C]    ← A printed, B is next
Print    → [C]       ← B printed, C is next
```

Why Queue? First submitted, first served. That's FIFO.

📁 Scenario 3

Your OS needs to represent folders containing files and other folders, which can contain more folders and files.

What ADT would you use?

What Doesn't Work Well

Array — a flat list can't represent folders inside folders. You'd need complex indexing hacks.

Stack / Queue — these model linear sequences, not hierarchical nesting.

Hash table — fast lookup, but no concept of *parent* and *child* relationships.

File System → Tree

Each folder is a **node**. Its contents are its **children**.

```
/home/
├── chike/
│   ├── Documents/
│   │   └── syllabus.pdf
│   └── Code/
│       └── lab1.cpp
└── shared/
    └── readings.pdf
```

Why Tree? Each item has exactly one parent and any number of children. Trees are exactly that structure.

🗺️ Scenario 4

A navigation app needs to find the shortest driving route between two cities, accounting for roads and distances.

What ADT would you use?

What Doesn't Work Well

Array / List — you could list roads, but there's no way to express that "Road A connects to Road B."

Tree — trees don't allow cycles. Roads clearly do form loops.

Stack / Queue — no concept of connections or distances between places.

Navigation → Graph

Cities are **nodes**. Roads are **edges**. Distances are **weights**.

```
 WWU ——5——— Walla Walla
  |                |
  8               12
  |                |
Kennewick ——3—— Richland
```

Dijkstra's algorithm (Unit 9) finds the shortest path.

Why Graph? Maps have arbitrary connections and cycles. Only a graph can model that.

🔍 Scenario 5

A spell checker must instantly tell you whether a word is in the dictionary for every keystroke, on 200,000 words.

What ADT would you use?

What Doesn't Work Well

Unsorted array — scan up to 200,000 words per keystroke. Way too slow.

Sorted array + binary search — O(log n), about 18 comparisons. Better, but still...

BST — also O(log n). ~18 comparisons per lookup.

Is 18 comparisons per keystroke acceptable? Maybe. But we can do *much* better.

Spell Checker → Hash Table

Each word is stored at an index computed from its characters.

Looking up `"algorithm"`:

1. Compute `hash("algorithm")` → index 47,823
2. Check position 47,823
3. Done. **One step.**

Why Hash Table? Lookup is O(1) — constant time regardless of dictionary size.

Two programs. Both correct. One takes 3 days. One takes 2 seconds.

Both sort the same 10 million records.

What could explain the difference? Does it matter which one you use?

Algorithm Efficiency

Both programs are "correct" — but only one is useful.

As data grows, the **efficiency** of your algorithms becomes the determining factor in whether your software is practical at all.

We need a way to talk about efficiency that doesn't depend on machine speed, language, or compiler. We need something **universal**.

How should we measure an algorithm's speed?

You could time it with a stopwatch. But is that a good measurement?

What problems can you think of with timing as a measure of efficiency?

Measuring Efficiency

Timing has problems:

- Depends on the **machine** — a faster computer gives different numbers.
- Depends on the **compiler, language, and CPU load**.
- Only tells you about the **specific input** you tested.

Instead, we count the **number of basic operations** the algorithm performs as a function of the input size `n`.

This is machine-independent and tells us how the algorithm *scales*.

🤔 Best case, worst case, or average case?

Searching for a name in a list of 1,000 students.

Best case: the name is first. 1 comparison.
Worst case: the name is last (or not there). 1,000 comparisons.

Which should we care about most when evaluating an algorithm? Why?

Best, Worst, and Average Case

- **Best case** — the luckiest possible input *(item is first)*
- **Worst case** — the hardest possible input *(item is last or absent)*
- **Average case** — what you'd expect on a typical random input

We usually focus on **worst case** because it's a *guarantee*: no matter what input you receive, the algorithm will perform no worse than this.

Best case is often uninteresting: a broken algorithm can have a great best case.

What happens to runtime as input size grows?

Algorithm A does `n` steps for `n` items.
Algorithm B does n^2 steps.

For `n = 10`, A does 10 steps and B does 100.

What happens when `n = 1,000`? When `n = 1,000,000`?

Introducing Big-O Notation

We use **Big-O notation** to express how runtime grows as `n` grows.

The key idea: we care about the *growth rate*, not the exact count. Constants and lower-order terms don't matter for large `n`.

Notation	Name	Example
O(1)	Constant	Array access by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, finding a max
O(n log n)	Log-linear	Merge sort, quicksort (avg)
$O(n^2)$	Quadratic	Bubble sort, selection sort
$O(2^n)$	Exponential	Naïve recursive Fibonacci

A Visual Intuition

n	O(log n)	O(n)	O(n log n)	$O(n^2)$
10	3	10	33	100
100	7	100	664	10,000
1,000	10	1,000	9,966	1,000,000
1,000,000	20	1,000,000	~20,000,000	10^{12}

At n = 1,000,000:

- O(log n) → **20 operations**
- O(n) → **1 million operations**
- $O(n^2)$ → **1 trillion operations** — roughly **11 days** at 1 billion ops/second

🤔 Is a faster algorithm always better?

Imagine Algorithm A runs in $O(n)$ time but uses $O(n^2)$ memory.

Algorithm B runs in $O(n^2)$ time but uses $O(1)$ memory.

On a machine with very limited RAM, which would you prefer?

Space vs. Time

Algorithms have two kinds of cost:

- **Time complexity** — how many operations does it perform?
- **Space complexity** — how much memory does it use?

These often trade off. A faster algorithm might need more memory. A memory-efficient algorithm might be slower.

Part of good algorithm design is understanding this tradeoff and choosing based on your **actual constraints**.

What We'll Study This Term

Over 10 weeks, you'll learn the most important data structures and the algorithms that work on them — and analyze why each is efficient using Big-O.

```
Arrays & basics → Sorting → Linked Lists → Stacks & Queues
     → Hash Tables → Heaps → Graphs → Trees → Advanced Trees
```

By the end, you'll be able to:

1. Choose the right data structure for a given problem.
2. Implement that structure from scratch in C++.
3. Analyze the time and space complexity of your solution.
4. Know when to use the standard library and when to roll your own.

Lab:
https://github.com/abuach/242students/tree/main/labs